Sure Here is a **simple Multiple Linear Regression** example with 2 independent variables.

Example

We want to predict **Salary** using:

- Experience
- Age

```
from sklearn.linear_model import LinearRegression

# Independent variables
X = [
    [1, 22],
    [2, 25],
    [3, 28],
    [4, 30],
    [5, 35]
]

# Dependent variable
y = [25000, 30000, 35000, 42000, 50000]

# Create model
model = LinearRegression()

# Train model
model.fit(X, y)

# Coefficients
print("Coefficients:", model.coef_)

# Intercept
print("Intercept:", model.intercept_)

# Predict salary for Experience=6, Age=37
prediction = model.predict([[6, 37]])

print("Predicted Salary:", prediction)
```

Multiple Linear Regression formula

y=mx+b
m (slope)
1.
m (slope)
b (y-intercept)
5.
b (y-intercept)
Give feedback

For multiple regression, it is:

$$y = b_0 + b_1X_1 + b_2X_2$$

For example:

Salary = Intercept + (Coefficient$_1$ × Experience) + (Coefficient$_2$ × Age)

Here:

- **y** → predicted salary
- b_0 → intercept
- b_1, b_2 → coefficients
- X_1, X_2 → input features

If you want, I can next show you **Multiple Linear Regression using a Pandas DataFrame + train_test_split + MAE + MSE + R²**, which is the more practical ML version.

Show me Multiple Linear Regression using a Pandas DataFrame with train_test_split, MAE, MSE, and R²

Absolutely. Here is a **practical Multiple Linear Regression example using Pandas DataFrame, `train_test_split`, MAE, MSE, and R²**.

1. Import libraries

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

2. Create a sample DataFrame

```
data = {
    "Experience": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "Age": [22, 24, 26, 28, 30, 32, 34, 36, 38, 40],
    "Education": [12, 13, 14, 15, 16, 16, 17, 18, 18, 19],
    "Salary": [25000, 28000, 32000, 36000, 40000,
               45000, 49000, 54000, 59000, 65000]
}

df = pd.DataFrame(data)

print(df)
```

3. Select X and y

```
X = df[["Experience", "Age", "Education"]]
```

```
y = df["Salary"]
```

Here:

- `X` = independent variables/features
- `y` = dependent variable/target

4. Split the data

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

`test_size=0.2` means **80% training** and **20% testing**.

5. Create and train the model

```
model = LinearRegression()

model.fit(X_train, y_train)
```

6. Make predictions

```
y_pred = model.predict(X_test)

print("Predicted:", y_pred)
print("Actual:", y_test.values)
```

7. Calculate MAE, MSE and R^2

```
mae = mean_absolute_error(y_test, y_pred)

mse = mean_squared_error(y_test, y_pred)

r2 = r2_score(y_test, y_pred)

print("MAE:", mae)
print("MSE:", mse)
print("R² Score:", r2)
```

8. Check coefficients and intercept

```
print("Coefficients:", model.coef_)
print("Intercept:", model.intercept_)
```

Your model is essentially learning:

y^=b0+b1x
y^=8.49−0.54x\hat{y} = 8.49 - 0.54xy^=8.49−0.54x
$R^2 = 0.72$ · b_0 = intercept · b_1 = slope · least squares minimizes squared vertical residual gaps.
Focus
DataLinePredictionsResiduals
DataLinePredictionsResiduals
Drag a point to refit the line.
246810246810
Give feedback

Salary = Intercept + (Coefficient$_1$ × Experience) + (Coefficient$_2$ × Age) + (Coefficient$_3$ × Education)

9. Predict a new person's salary

For example:

- Experience = 7 years
- Age = 35
- Education = 17 years

```
new_data = pd.DataFrame({
    "Experience": [7],
    "Age": [35],
    "Education": [17]
})

prediction = model.predict(new_data)

print("Predicted Salary:", prediction[0])
```

Remember the evaluation metrics

Metric	Meaning	Better
MAE	Average absolute prediction error	Lower
MSE	Average squared prediction error	Lower
R^2	How well the model explains the target	Closer to 1

Example: If MAE = `2000`, your predictions are off by about **₹2,000 on average**.